Machine Learning, Logistic Regression

Dr. Ahmed Aljarah

Classification

- It involves splitting a dataset into two groups: those that satisfy a condition and those that do not.

- Examples include:
  - Email: Spam / Not Spam
  - Exam: Pass / Fail
  - Tumors: Malignant / Benign

- The output values in the training data are always {0, 1}, where $y \in \{0,1\}$

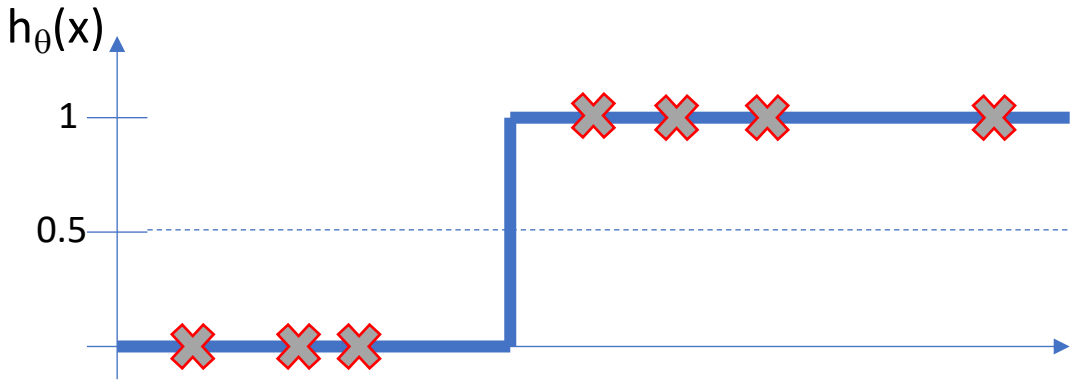
- Here, 0 represents the negative class and 1 represents the positive class.

Logistic Regression

- Logistic Regression is a supervised machine learning algorithm used for classification tasks.
- It estimates the probability that an input belongs to a specific class.
- It is mainly used for binary classification problems, where the output has two possible categories such as (Yes, No), (True, False), or (0, 1).
- It applies a sigmoid function to transform the input into a probability value between 0 and 1.

Classification threshold

- It is the value used to decide which class a prediction belongs to based on its probability.
- The output $h_\theta(x)$ is a number between 0 and 1 (a probability).
- The threshold is the cutoff point used to convert that probability into a final class label.
- The classification threshold is therefore 0.5:
  If $h_\theta(x) \geq 0.5$, predict "y = 1"
  If $h_\theta(x) < 0.5$, predict "y = 0"
- The ideal function would be a step function.



Sigmoid function

- It is used to convert any real number into a value between 0 and 1, making it perfect for representing probabilities.

$$0 \leq h_\theta(x) \leq 1$$

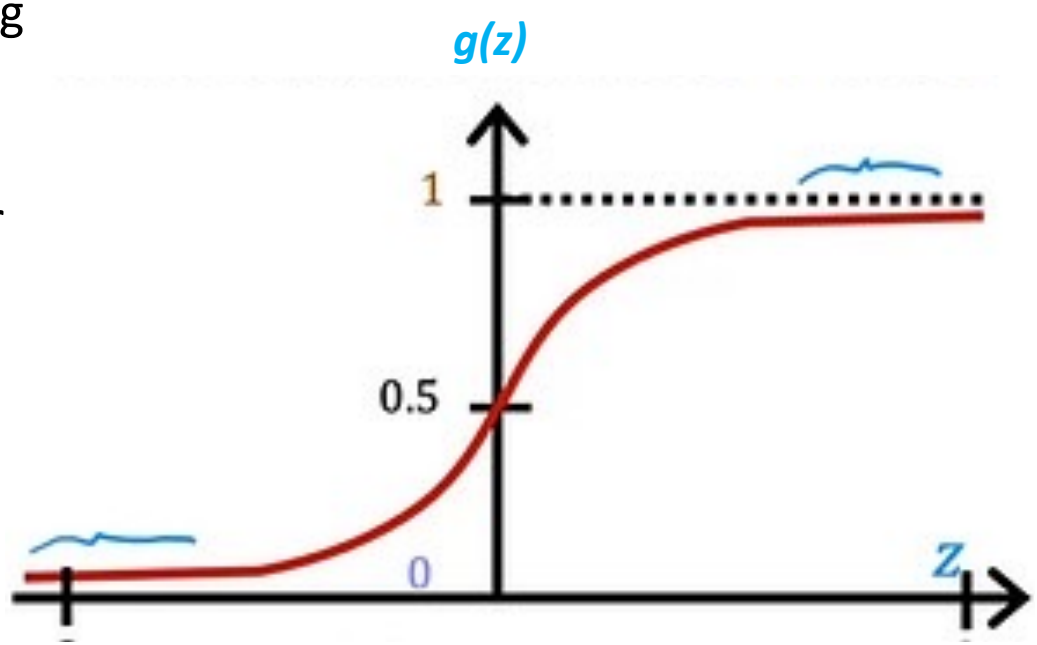
- First, the model computes a value z (a linear combination of the input features):

$$z = \theta^T x$$

- Then, it applies the sigmoid function:

$$g(z) = \frac{1}{1+e^{-z}}$$

- Where e = euler's number 2.71828



Formulas

$$h_\theta(x) = g(z) = g(\theta^T X) = \frac{1}{1 + e^{-\theta^T X}}$$

- Where $\theta^T x$ = a linear combination of the input features
- Where T means transpose: $z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_n x_n$
- g(z): is the sigmoid function, which converts any value into a number between 0 and 1

Logistic Regression works

1. Computes a linear value: $z=\theta^T x$

2. Sigmoid function is applied: $h_\theta(x)=g(z) = \frac{1}{1+e^{-\theta^T X}}$

3. Interpretation
   - Result is transformed into a probability.
   - $h_\theta(x)$ can therefore be interpreted as the probability that y=1:
   - $h_\theta(x)=P(y=1|x;\theta)$ that is, the probability that y=1 given x, parameterized by θ

4. Simple intuition
   - If $h\theta(x)\approx 1 \rightarrow$ very likely class 1
   - If $h\theta(x)\approx 0 \rightarrow$ very likely class 0

Example:

Suppose you are predicting whether an email is spam (1) or not spam (0).

- If the model computes z=2: then

$$g(2) = \frac{1}{1+e^{-z}} = \frac{1}{1+e^{-2}} \simeq 0.88$$

- That means, 88% probability that the email is spam and 12% probability that the email is not spam.

- It turns any output into a probability, allowing the model to make decisions like:
  - If $h\theta(x) \geq 0.5 \rightarrow$ classify as 1
  - If $h\theta(x) < 0.5 \rightarrow$ classify as 0

Odds ratio

- Odds measure how likely an event is to happen compared to it not happening.

$$\text{Odds}=\frac{P}{1-P}$$

Where:

- P: probability that the event happens
- 1−P: probability that it does NOT happen

Example 1:
P=0.8

$$\text{Odds}=\frac{0.8}{1-0.8}=\frac{0.8}{0.2}=4$$

The event is 4 times more likely to happen than not

Example 2:
P=0.5

$$\text{Odds}=\frac{0.5}{1-0.5}=\frac{0.5}{0.5}=1$$

Equal chance (50–50)

Example 3:
P=0.2

$$\text{Odds}=\frac{0.2}{1-0.2}=\frac{0.2}{0.8}=0.25$$

The event is less likely to happen than not

Log-Odds (Logit)

$\log(\frac{p}{1-p})=z$

- P= $\frac{1}{1+e^{-z}}$ →
- Invers both side: $\frac{1}{p} = 1 + e^{-z}$ →
- $\frac{1}{p}$ - 1 = e^{-z} →
- Simplify left side: $\frac{1-p}{p} = e^{-z}$ →
- Flip both side: $\frac{p}{1-p} = e^{z}$ → $\log(\frac{p}{1-p})=z$

- Where: P=P(y=1|x) and $z=\theta^t x$
- The model is linear in log-odds, not in probability
- Probability modeling, for each data point:

$$P(y \mid x) = \begin{cases} p & if\ y = 1 \\ 1-p & if\ y = 0 \end{cases}$$

Then : probability of correct label = $p^y (1-p)^{1-y}$

Cost funtion in logistic regression

- The **cost function** measures how well the model's predictions match the actual values.

$h_\theta(x)=P(y=1|x)$

$$Costo(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & if\ y = 1 \\ -\log(1 - h_\theta(x)) & if\ \ y = 0 \end{cases}$$

The log function has a special behavior:

- If y =1 and $h_\theta(x) = 0.9$

  then -log(0.9)≈0.1 → small error
- If y =1 and $h_\theta(x) = 0.01$

  Then -log(0.01) ≈ 4.6 → huge penalty
- The model is slightly penalized when it's correct
- It is heavily penalized when it's wrong

Cost funtion simplify

- Since y is always {0, 1}, it can be used as a binary variable to simplify the function into a single expression:

$$Costo(h_\theta(x), y) = -ylog\big(h_\theta(x)\big) - (1-y)\log(1-h_\theta(x))$$

- $h_\theta(x)$ = predicted probability
- Y= actual value (0 or 1)

Two cases:

- If y = 1, Cost=$-\log(h_\theta(x))$
  - If prediction is close to 1, then cost is small
  - If prediction is close to 0 , then cost is very large
- If y=0, Cost=$-\log(1-h_\theta(x))$
  - If prediction is close to 0, then cost is small
  - If prediction is close to 1, then cost is very large

Cost funtion simplify

$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n} Cost(h_\theta(x^{(i)}), y^{(i)})$$

$$= -\frac{1}{n}\left[\sum_{i=1}^{n} y^{(i)} log\left(h_\theta\left(x^{(i)}\right)\right) + \left(1 - y^{(i)}\right)\log(1 - h_\theta\left(x^{(i)}\right))\right]$$

- Compute the cost for each example
- Then take the average over all m examples
- The function measures how far predictions are from the real labels, giving:
  - Low cost → good predictions
  - High cost → bad predictions
- The goal is to find θ that minimizes cost function: j(θ)

Regularization Lasso L1

$$j(\theta)=-\frac{1}{n}\left[\sum_{i=1}^{n} y^{(i)} log\left(h_\theta\left(x^{(i)}\right)\right)+\left(1-y^{(i)}\right)\log\left(1-h_\theta\left(x^{(i)}\right)\right)\right]+\lambda\sum_{j=1}^{p}|\theta_j|$$

$$\theta_j=\theta_j-\alpha\,.\,\frac{\partial J}{\partial\theta_j}$$

$$\theta_j=\theta_j-\alpha\,.\,[\frac{2}{n}\sum(h_\theta(x_i)-y_i)x_{ij}+\lambda\cdot sign(\theta_j)]$$

Regularization Ridge L2

$$j(\theta)=-\frac{1}{n}\left[\sum_{i=1}^{n} y^{(i)} log\left(h_\theta\left(x^{(i)}\right)\right)+\left(1-y^{(i)}\right)\log\left(1-h_\theta\left(x^{(i)}\right)\right)\right]+\lambda\sum_{j=1}^{p}\theta_j^2$$

$$\theta_j=\theta_j-\alpha\,.\,\frac{\partial J}{\partial\theta_j}$$

$$\theta_j=\theta_j-\alpha\,.\,[\frac{2}{n}\sum(h_\theta(x_i)-y_i)x_{ij}+\lambda\sum_{j=1}^{p}\theta_j^2]$$

Accuracy

- Accuracy measures how many predictions the model got correct out of all predictions.

$$\text{Accuracy} = \frac{True\ Positive + True\ Negative}{Total}$$

  - True Positives (TP): Model correctly predicts positive cases
  - True Negatives (TN): Model correctly predicts negative cases
  - Total: All predictions

Example : TP = 40 , TN = 50 and Total = 100

- Accuracy = $\frac{40+50}{100}$ = 0.90 Accuracy
- Accuracy = **90%**

Multiclass Logistic Regression

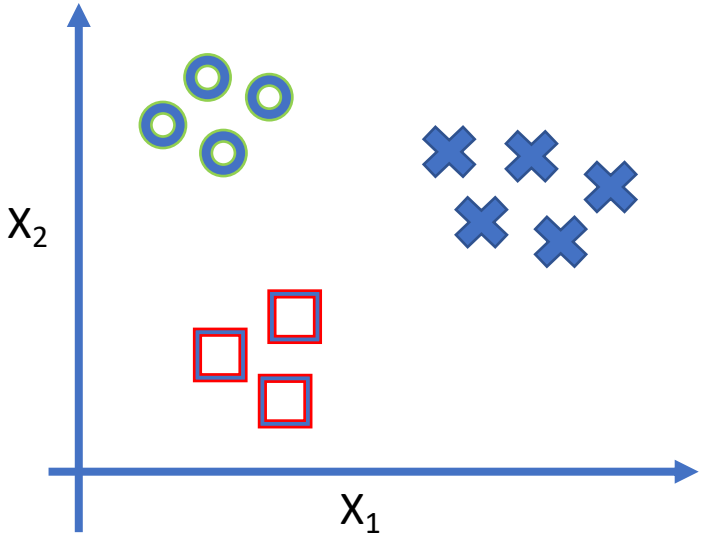
- It is used when the dependent variable has three or more categories with no inherent order.

For example, classifying animals as:

- Cat : y =1
- Dog : y =2
- Sheep: y =3

- It is an extension of binary logistic regression

that allows the model to handle multiple classes.

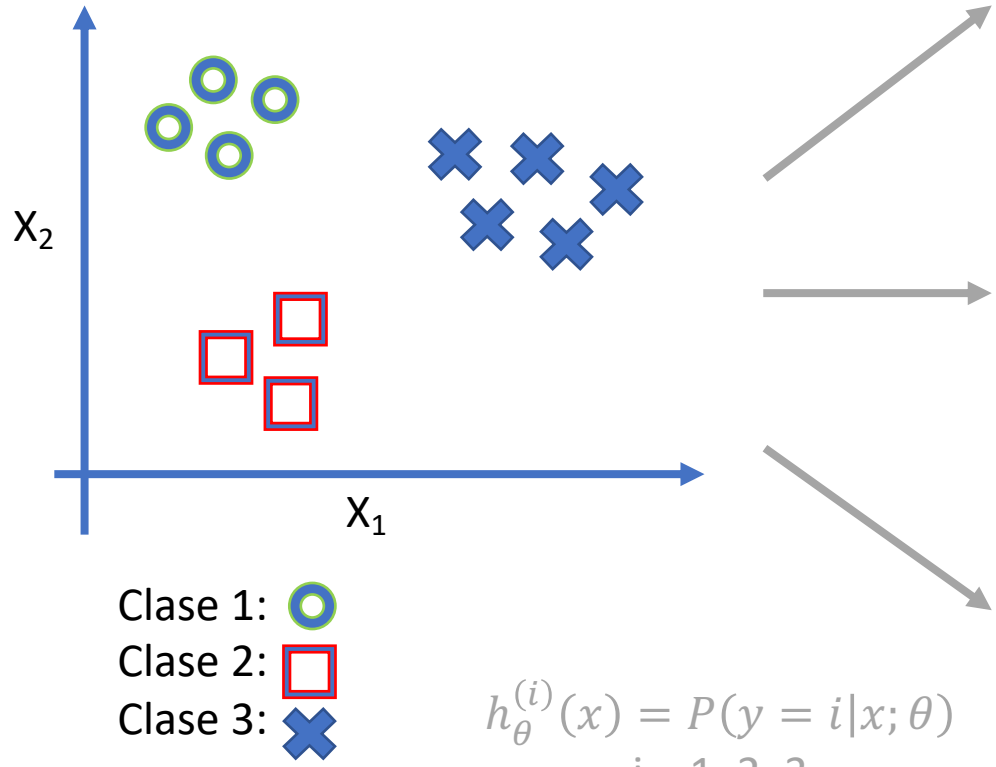


Clase 1:
Clase 2:
Clase 3:

$$h_\theta^{(i)}(x) = P(y = i|x; \theta)$$

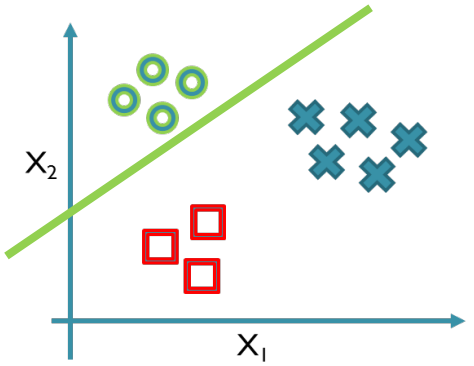
i = 1, 2, 3

Graph

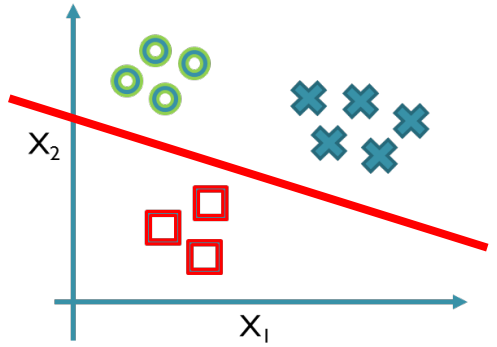


$h_\theta^{(i)}(x) = P(y = i|x;\theta)$

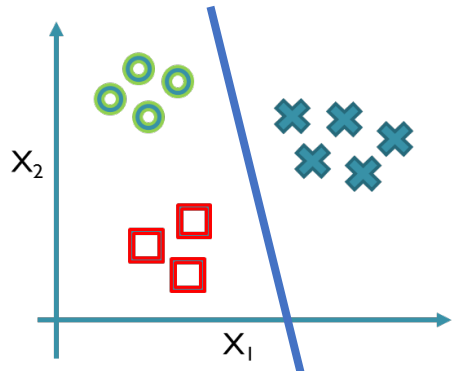
i = 1, 2, 3



$h_\theta^{(0)}(x) = P(y = 1|x;\theta)$



$h_\theta^{(1)}(x) = P(y = 2|x;\theta)$



$h_\theta^{(2)}(x) = P(y = 3|x;\theta)$

Example 1:

Predict digits from images using a logistic regression classifier in python

```
#Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits

#Download the dataset
images=load_digits()
images.data.shape
images.target.shape

#Show five images
plt.figure(figsize=(20,12))
for index,(img,label) in
enumerate(zip(images.data[0:5],images.target[0:5])):
plt.subplot(1,5,index+1)
plt.imshow(np.reshape(img,(8,8)),cmap= plt.cm.gray)
plt.title(f"Training {label}")
```

```
#Separate dataset
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test =
train_test_split(images.data,images.target,test_size=0.2,random_state=100)

#Create the model
from sklearn.linear_model import LogisticRegression
logistic = LogisticRegression(C = 0.1, max_iter=300)
logistic.fit(x_train,y_train)

#Model evaluation
y_pred = logistic.predict(x_test)
from sklearn.metrics import accuracy_score, confusion_matrix
a = accuracy_score(y_test,y_pred)
print("Accuracy: ",a)
cm = confusion_matrix(y_test,y_pred)
print(cm)
plt.figure(figsize=(10,10))
import seaborn as sns
sns.heatmap(cm,annot=True, linewidths = 5,cmap="Blues_r")
plt.ylabel("Actual labels")
plt.xlabel("Predicted labels")
plt.title(f"Model's accuracy {a}",size=15)
```

Example 2:

Predict whether a patient has diabetes using a logistic regression classifier.

```
#Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

#Create data frame from the dataset
df = pd.read_csv("/content/drive/MyDrive/AI course/diabetes_data_upload.csv",
sep=",")
df.head()

#Data analysis
df.shape
df.isna().sum()
df.nunique()
df["Gender"].unique()

#Convert categorical feature to numerical
g = {"Male":1,"Female": 0}
df["Gender"]= df["Gender"].map(g)
```

```
 #Separate the dataset to input and out put
 x = df.iloc[:,:-1]
 y = df.iloc[:,-1]
 y.head()

#Separate dataset to train and test samples
from sklearn.model_selection import train_test_split
x_train,x_test,y_train, y_test =
train_test_split(x,y,test_size=0.2,random_state=100)


 #Create the model
 from sklearn.linear_model import LogisticRegression
 logistic = LogisticRegression(C=0.1,max_iter=1000)
 logistic.fit(x_train,y_train)

 #Model evaluation
 y_pred = logistic.predict(x_test)
 from sklearn.metrics import accuracy_score
 print(f"Model's accuracy : {accuracy_score(y_test,y_pred)}")
```